

In [29]: `pip install pandas`

Requirement already satisfied: pandas in c:\users\sahil\anaconda3\lib\site-packages (2.3.3)
 Requirement already satisfied: numpy>=1.26.0 in c:\users\sahil\anaconda3\lib\site-packages (from pandas) (2.3.5)
 Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\sahil\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in c:\users\sahil\anaconda3\lib\site-packages (from pandas) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in c:\users\sahil\anaconda3\lib\site-packages (from pandas) (2025.2)
 Requirement already satisfied: six>=1.5 in c:\users\sahil\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
 Note: you may need to restart the kernel to use updated packages.

In [30]: `pip install numpy`

Requirement already satisfied: numpy in c:\users\sahil\anaconda3\lib\site-packages (2.3.5)
 Note: you may need to restart the kernel to use updated packages.

In [31]: `pip install seaborn`

Requirement already satisfied: seaborn in c:\users\sahil\anaconda3\lib\site-packages (0.13.2)
 Requirement already satisfied: numpy!=1.24.0,>=1.20 in c:\users\sahil\anaconda3\lib\site-packages (from seaborn) (2.3.5)
 Requirement already satisfied: pandas>=1.2 in c:\users\sahil\anaconda3\lib\site-packages (from seaborn) (2.3.3)
 Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in c:\users\sahil\anaconda3\lib\site-packages (from seaborn) (3.10.6)
 Requirement already satisfied: contourpy>=1.0.1 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
 Requirement already satisfied: cycler>=0.10 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)
 Requirement already satisfied: fonttools>=4.22.0 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.60.1)
 Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.9)
 Requirement already satisfied: packaging>=20.0 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (25.0)
 Requirement already satisfied: pillow>=8 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.0.0)
 Requirement already satisfied: pyparsing>=2.3.1 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.2.5)
 Requirement already satisfied: python-dateutil>=2.7 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in c:\users\sahil\anaconda3\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in c:\users\sahil\anaconda3\lib\site-packages (from pandas>=1.2->seaborn) (2025.2)
 Requirement already satisfied: six>=1.5 in c:\users\sahil\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
 Note: you may need to restart the kernel to use updated packages.

In [32]: `pip install matplotlib`

Requirement already satisfied: matplotlib in c:\users\sahil\anaconda3\lib\site-packages (3.10.6)
 Requirement already satisfied: contourpy>=1.0.1 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (1.3.3)
 Requirement already satisfied: cycler>=0.10 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
 Requirement already satisfied: fonttools>=4.22.0 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (4.60.1)
 Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (1.4.9)
 Requirement already satisfied: numpy>=1.23 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (2.3.5)
 Requirement already satisfied: packaging>=20.0 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (25.0)
 Requirement already satisfied: pillow>=8 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (12.0.0)
 Requirement already satisfied: pyparsing>=2.3.1 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (3.2.5)
 Requirement already satisfied: python-dateutil>=2.7 in c:\users\sahil\anaconda3\lib\site-packages (from matplotlib) (2.9.0.post0)
 Requirement already satisfied: six>=1.5 in c:\users\sahil\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
 Note: you may need to restart the kernel to use updated packages.

```
In [33]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [34]: df = pd.read_csv('ipl_2026_deliveries.csv')
df.head()
```

```
Out[34]:
```

	match_id	season	phase	match_no	date	venue	batting_team	bowling_te
0	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
1	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
2	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
3	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
4	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	

5 rows × 21 columns



```
In [35]: df.describe()
```

Out[35]:

	match_id	season	match_no	innings	over	runs_of_bat
count	17477.000000	17477.0	17477.000000	17477.000000	17477.000000	17477.000000
mean	202637.987927	2026.0	37.987927	1.480002	9.478646	1.496424
std	21.299799	0.0	21.299799	0.500301	5.649094	1.852155
min	202601.000000	2026.0	1.000000	1.000000	0.100000	0.000000
25%	202620.000000	2026.0	20.000000	1.000000	4.500000	0.000000
50%	202638.000000	2026.0	38.000000	1.000000	9.400000	1.000000
75%	202657.000000	2026.0	57.000000	2.000000	14.300000	2.000000
max	202674.000000	2026.0	74.000000	4.000000	19.600000	6.000000

In [36]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17477 entries, 0 to 17476
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   match_id              17477 non-null  int64
1   season                17477 non-null  int64
2   phase                 17477 non-null  object
3   match_no              17477 non-null  int64
4   date                  17477 non-null  object
5   venue                 17477 non-null  object
6   batting_team          17477 non-null  object
7   bowling_team          17477 non-null  object
8   innings               17477 non-null  int64
9   over                  17477 non-null  float64
10  striker               17477 non-null  object
11  bowler                17477 non-null  object
12  runs_of_bat           17477 non-null  int64
13  extras                17477 non-null  int64
14  wide                  17477 non-null  int64
15  legbyes               17477 non-null  int64
16  byes                  17477 non-null  int64
17  noballs               17477 non-null  int64
18  wicket_type           879 non-null    object
19  player_dismissed      875 non-null    object
20  fielder               686 non-null    object
dtypes: float64(1), int64(10), object(10)
memory usage: 2.8+ MB
```

In [37]: `df.shape`

Out[37]: (17477, 21)

In [38]: `print(df.columns)`

```
Index(['match_id', 'season', 'phase', 'match_no', 'date', 'venue',
      'batting_team', 'bowling_team', 'innings', 'over', 'striker', 'bowler',
      'runs_of_bat', 'extras', 'wide', 'legbyes', 'byes', 'noballs',
      'wicket_type', 'player_dismissed', 'fielder'],
      dtype='object')
```

```
In [39]: df.isnull().sum()
```

```
Out[39]: match_id          0
season          0
phase           0
match_no        0
date            0
venue           0
batting_team     0
bowling_team     0
innings          0
over            0
striker          0
bowler           0
runs_of_bat      0
extras           0
wide            0
legbyes          0
byes             0
noballs          0
wicket_type      16598
player_dismissed 16602
fielder          16791
dtype: int64
```

```
In [40]: df['match_id'].unique()
```

```
Out[40]: array([202601, 202602, 202603, 202604, 202605, 202606, 202607, 202608,
                202609, 202610, 202611, 202612, 202613, 202614, 202615, 202616,
                202617, 202618, 202619, 202620, 202621, 202622, 202623, 202624,
                202625, 202626, 202627, 202628, 202629, 202630, 202631, 202632,
                202633, 202634, 202635, 202636, 202637, 202638, 202639, 202640,
                202641, 202642, 202643, 202644, 202645, 202646, 202647, 202648,
                202649, 202650, 202651, 202652, 202653, 202654, 202655, 202656,
                202657, 202658, 202659, 202660, 202661, 202662, 202663, 202664,
                202665, 202666, 202667, 202668, 202669, 202670, 202671, 202672,
                202673, 202674])
```

```
In [41]: df['match_id'].nunique()
```

```
Out[41]: 74
```

```
In [42]: df['batting_team'].unique()
```

```
Out[42]: array(['SRH', 'RCB', 'KKR', 'MI', 'CSK', 'RR', 'GT', 'PBKS', 'LSG', 'DC'],
               dtype=object)
```

```
In [43]: df['batting_team'].nunique()
```

```
Out[43]: 10
```

create new column

```
In [44]: df['Total_runs'] = df['runs_of_bat']+df['extras']

df['is_six'] = 0
df['is_four'] =0
df['is_wicket']= 0
df.loc[df['wicket_type'].notnull(),'is_wicket']=1

df.loc[df['runs_of_bat'] ==6 , 'is_six' ] = 1
df.loc[df['runs_of_bat'] ==4 , 'is_four' ] = 1
df.head()
```

Out[44]:

	match_id	season	phase	match_no	date	venue	batting_team	bowling_te
--	----------	--------	-------	----------	------	-------	--------------	------------

0	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
1	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
2	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
3	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
4	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	

5 rows × 25 columns



```
In [45]: df['wicket_type'].unique()
```

Out[45]: array([nan, 'caught', 'bowled', 'runout', 'lbw', 'stumped',
'retired hurt', 'obstructing the field', 'hitwicket', 'hit wicket'],
dtype=object)

```
In [46]: df['over'] = df['over'] +1 # change the starting over from 0 to 1
df.head()
```

Out[46]:

	match_id	season	phase	match_no	date	venue	batting_team	bowling_te
0	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
1	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
2	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
3	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
4	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	

5 rows × 25 columns



In [47]:

```
def get_phase(over):
    if over <= 6:
        return 'powerplay (1-6)'
    elif over <= 15:
        return 'middle (7-15)'
    else:
        return 'death (16-20)'

df['phase_group'] = df['over'].apply(get_phase)
df.head()
```

Out[47]:

	match_id	season	phase	match_no	date	venue	batting_team	bowling_te
0	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
1	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
2	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
3	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	
4	202601	2026	Group Stage	1	Mar 28, 2026	M.Chinnaswamy Stadium, Bengaluru	SRH	

5 rows × 26 columns



batting analysis

most run by team

In [48]: `df.columns`

```
Out[48]: Index(['match_id', 'season', 'phase', 'match_no', 'date', 'venue',
               'batting_team', 'bowling_team', 'innings', 'over', 'striker', 'bowler',
               'runs_of_bat', 'extras', 'wide', 'legbyes', 'byes', 'noballs',
               'wicket_type', 'player_dismissed', 'fielder', 'Total_runs', 'is_six',
               'is_four', 'is_wicket', 'phase_group'],
              dtype='object')
```

```
In [51]: # Group by batting team and sum total runs
edf = (
    df.groupby('batting_team', as_index=False)
      .agg(total=('Total_runs', 'sum'))
      .sort_values('total', ascending=False)
      .reset_index(drop=True)
)
edf
```

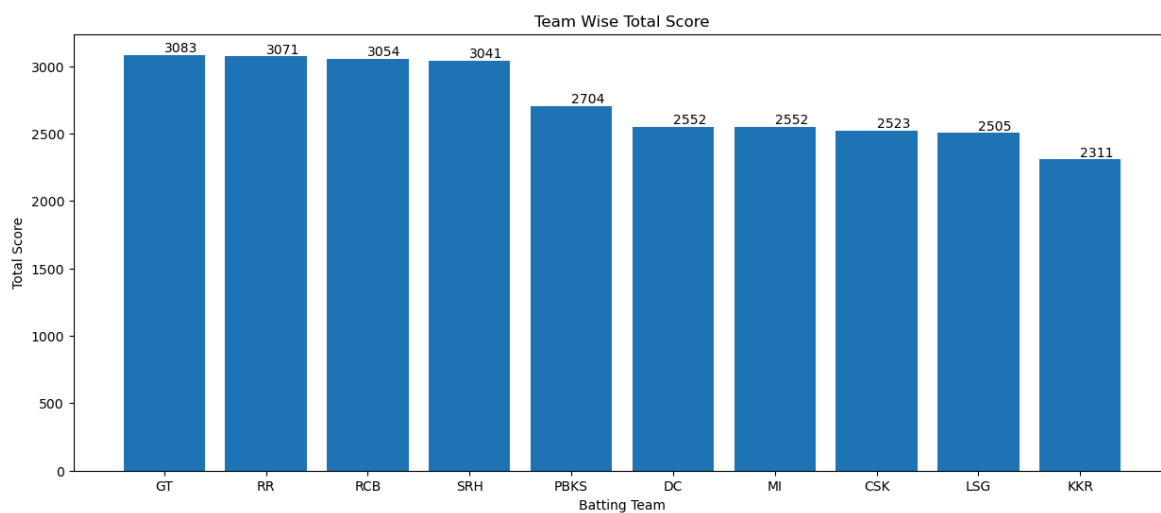
Out[51]:

	batting_team	total
0	GT	3083
1	RR	3071
2	RCB	3054
3	SRH	3041
4	PBKS	2704
5	DC	2552
6	MI	2552
7	CSK	2523
8	LSG	2505
9	KKR	2311

```
In [59]: plt.figure(figsize=(15,6))
plt.bar(edf['batting_team'], edf['total'])
plt.title('Team Wise Total Score')
plt.xlabel('Batting Team')
plt.ylabel('Total Score')

for x, y in zip(edf['batting_team'], edf['total']):
    plt.text(x=x, y=y, s=y, va='bottom',)

plt.show()
```



team sixes and four

```
In [60]: edf = df.groupby('batting_team').agg(
    nos=('is_six', 'sum'),
    nof=('is_four', 'sum')
).reset_index()

edf['score by boundaries'] = edf['nos'] + edf['nof']
edf
```


Out[60]:

	batting_team	nos	nof	score by boundaries
0	CSK	124	220	344
1	DC	117	225	342
2	GT	124	292	416
3	KKR	115	192	307
4	LSG	127	208	335
5	MI	140	204	344
6	PBKS	163	219	382
7	RCB	164	261	425
8	RR	181	257	438
9	SRH	171	255	426

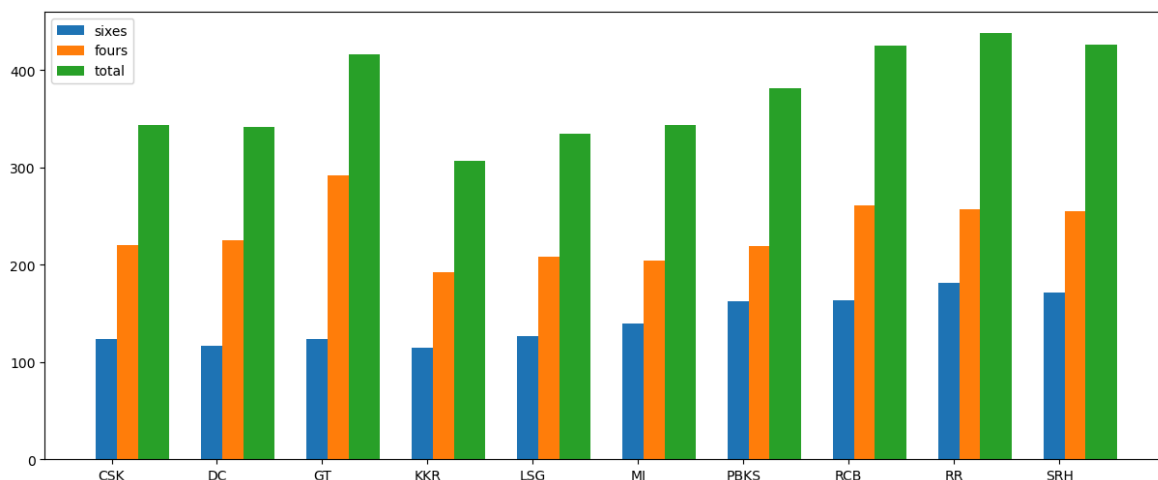
```
In [61]: plt.figure(figsize=(15,6))

xval = np.arange(1,11)

plt.bar(xval, edf['nos'], width=0.3, label='sixes')
plt.bar(xval+0.2, edf['nof'], width=0.3, label='fours')
plt.bar(xval+0.4, edf['score by boundaries'], width=0.3, label='total')

plt.legend()

plt.xticks(xval, edf['batting_team'])
plt.show()
```



phase wise team run

```
In [64]: edf = df.groupby(['batting_team', 'phase_group']).agg(
            total=('Total_runs', 'sum')
        ).reset_index()

edf
```

Out[64]:

	batting_team	phase_group	total
0	CSK	death (16-20)	758
1	CSK	middle (7-15)	1060
2	CSK	powerplay (1-6)	705
3	DC	death (16-20)	792
4	DC	middle (7-15)	1136
5	DC	powerplay (1-6)	624
6	GT	death (16-20)	830
7	GT	middle (7-15)	1429
8	GT	powerplay (1-6)	824
9	KKR	death (16-20)	635
10	KKR	middle (7-15)	1069
11	KKR	powerplay (1-6)	607
12	LSG	death (16-20)	746
13	LSG	middle (7-15)	1065
14	LSG	powerplay (1-6)	694
15	MI	death (16-20)	758
16	MI	middle (7-15)	1191
17	MI	powerplay (1-6)	603
18	PBKS	death (16-20)	813
19	PBKS	middle (7-15)	1176
20	PBKS	powerplay (1-6)	715
21	RCB	death (16-20)	861
22	RCB	middle (7-15)	1315
23	RCB	powerplay (1-6)	878
24	RR	death (16-20)	782
25	RR	middle (7-15)	1404
26	RR	powerplay (1-6)	885
27	SRH	death (16-20)	839
28	SRH	middle (7-15)	1393
29	SRH	powerplay (1-6)	809

```
In [66]: edf = df.pivot_table(index='batting_team', columns='phase_group',
                             values='Total_runs', aggfunc='sum').reset_index()
edf
```

Out[66]:

phase_group	batting_team	death (16-20)	middle (7-15)	powerplay (1-6)
0	CSK	758	1060	705
1	DC	792	1136	624
2	GT	830	1429	824
3	KKR	635	1069	607
4	LSG	746	1065	694
5	MI	758	1191	603
6	PBKS	813	1176	715
7	RCB	861	1315	878
8	RR	782	1404	885
9	SRH	839	1393	809

In [69]: `edf.columns`

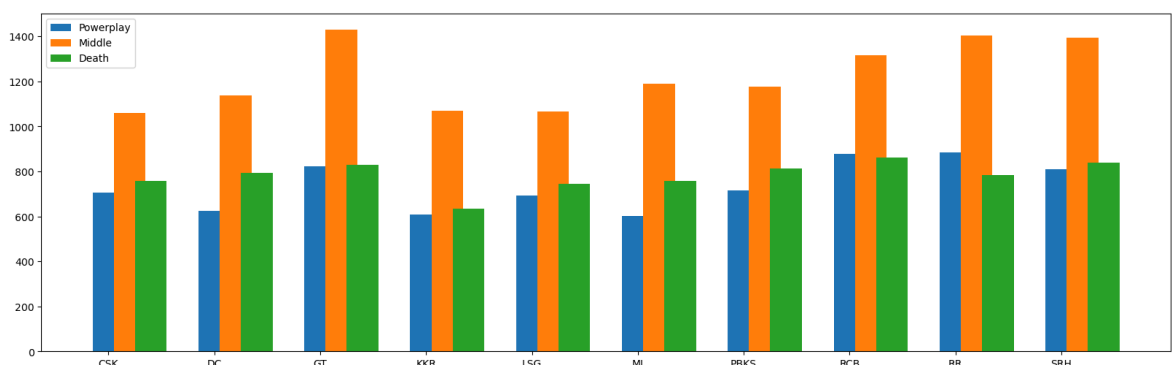
Out[69]: Index(['batting_team', 'death (16-20)', 'middle (7-15)', 'powerplay (1-6)'], dtype='object', name='phase_group')

In [73]: `plt.figure(figsize=(20, 6))`

`xval = np.arange(1, 11)`

`plt.bar(xval, edf['powerplay (1-6)'], width=0.3, label='Powerplay')`
`plt.bar(xval+0.2, edf['middle (7-15)'], width=0.3, label='Middle')`
`plt.bar(xval+0.4, edf['death (16-20)'], width=0.3, label='Death')`

`plt.legend()`
`plt.xticks(xval, edf['batting_team'])`
`plt.show()`



top 10 run score

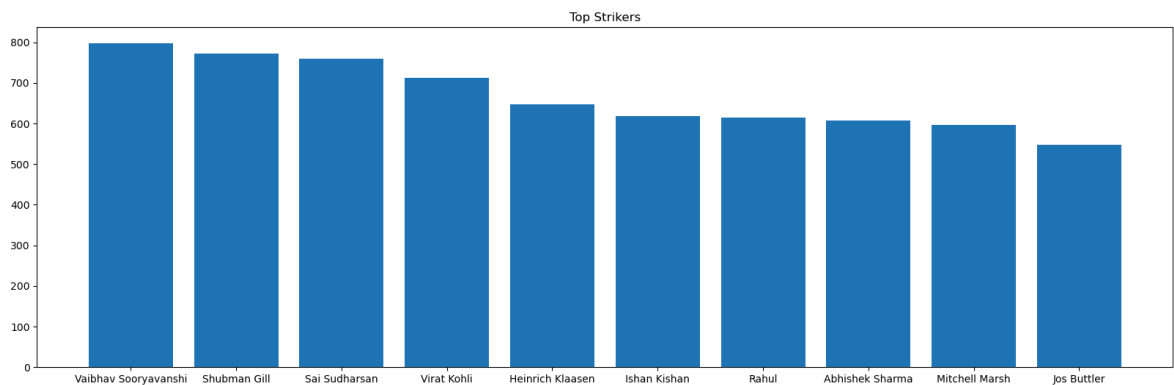
In [76]: `edf = df.groupby('striker').agg(
total=('Total_runs', 'sum')
).reset_index()`

`edf = edf.sort_values('total', ascending=False).head(10)`
`edf`

Out[76]:

	striker	total
166	Vaibhav Sooryavanshi	797
149	Shubman Gill	772
132	Sai Sudharsan	760
171	Virat Kohli	712
51	Heinrich Klaasen	647
53	Ishan Kishan	618
112	Rahul	614
2	Abhishek Sharma	608
85	Mitchell Marsh	597
62	Jos Buttler	548

```
In [77]: plt.figure(figsize=(20,6))
plt.bar(edf['striker'], edf['total'])
plt.title('Top Strikers')
plt.show()
```



strike rate

```
In [78]: b = 'is_four'
edf = df.groupby('striker').agg(
    total=(b, 'sum')
).reset_index()

edf = edf.sort_values('total', ascending=False).head(10)
edf
```

Out[78]:

	striker	total
132	Sai Sudharsan	75
149	Shubman Gill	74
171	Virat Kohli	73
166	Vaibhav Sooryavanshi	63
53	Ishan Kishan	60
112	Rahul	56
105	Prabhsimran Singh	55
136	Sanju Samson	53
176	Yashasvi Jaiswal	52
85	Mitchell Marsh	51

In [97]:

```
# Exclude wide balls from balls faced
legal_balls_bat = df.loc[df['wide'] == 0]

edf_sr = legal_balls_bat.groupby('striker').agg(
    runs = ('runs_of_bat', 'sum'),
    balls = ('runs_of_bat', 'count')
).reset_index()

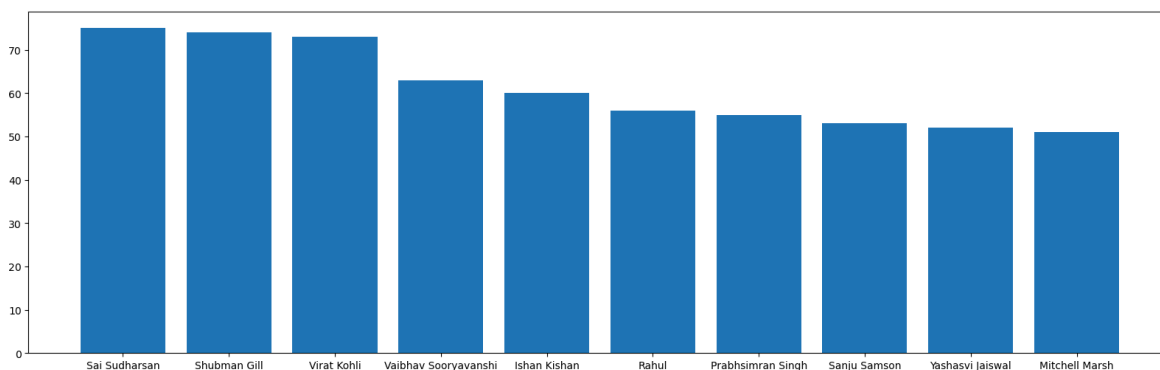
edf_sr = edf_sr.loc[edf_sr['balls'] >= 30]
edf_sr['strike_rate'] = ((edf_sr['runs'] / edf_sr['balls']) * 100).round().astype(int)
edf_sr = edf_sr.sort_values('strike_rate', ascending=False).head(10)
edf_sr
```

Out[97]:

	striker	runs	balls	strike_rate
166	Vaibhav Sooryavanshi	776	327	237
44	Finn Allen	349	163	214
110	Priyansh Arya	364	172	212
2	Abhishek Sharma	563	275	205
164	Urvil Patel	129	64	202
115	Rajat Patidar	500	259	193
142	Shashank Singh	132	70	189
158	Tim David	305	162	188
130	Ryan Rickelton	448	240	187
168	Venkatesh Iyer	209	112	187

In [79]:

```
plt.figure(figsize=(20,6))
plt.bar(edf['striker'], edf['total'])
plt.show()
```



top wicket taker

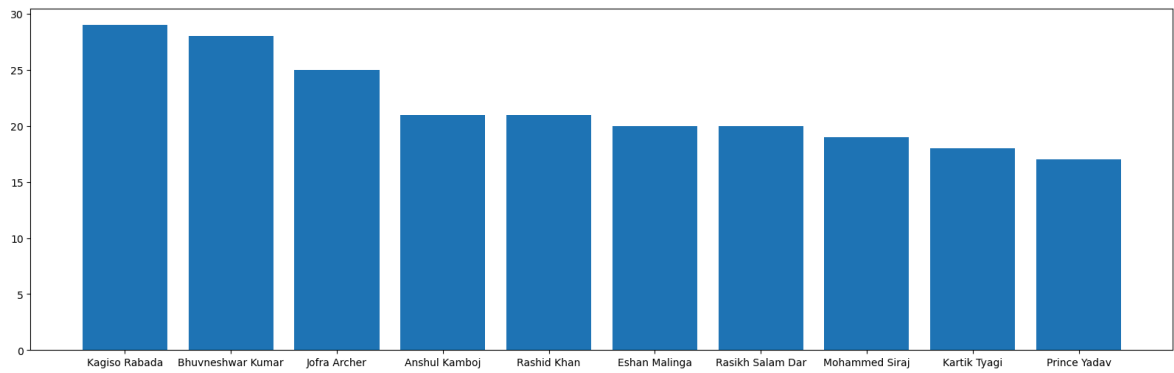
```
In [80]: bowler_wickets = df[(df['wicket_type'].notnull()) & (df['wicket_type']!='runout')
edf = bowler_wickets.groupby('bowler').agg(
    total=('is_wicket', 'sum')
).reset_index()

edf = edf.sort_values('total', ascending=False).head(10)
edf
```

```
Out[80]:
```

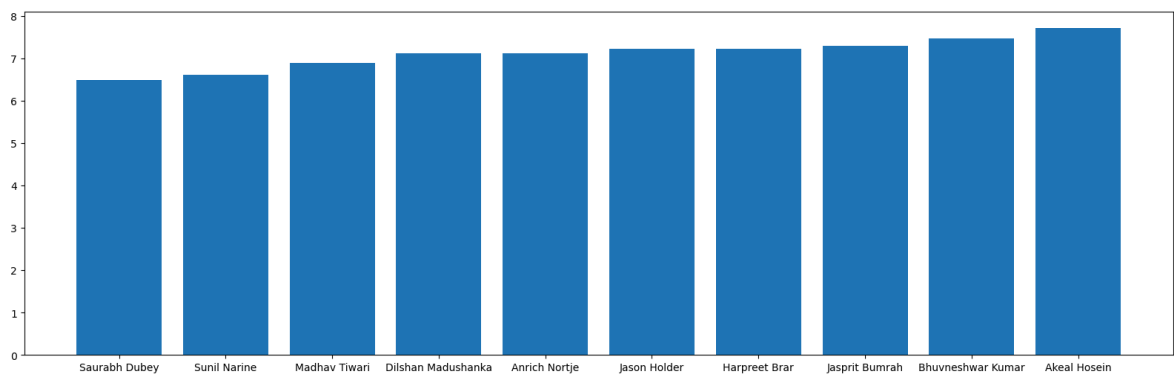
	bowler	total
40	Kagiso Rabada	29
15	Bhuvneshwar Kumar	28
38	Jofra Archer	25
4	Anshul Kamboj	21
73	Rashid Khan	21
26	Eshan Malinga	20
74	Rasikh Salam Dar	20
57	Mohammed Siraj	19
41	Kartik Tyagi	18
68	Prince Yadav	17

```
In [81]: plt.figure(figsize=(20,6))
plt.bar(edf['bowler'], edf['total'])
plt.show()
```



economy rate

```
In [83]: plt.figure(figsize=(20,6))
plt.bar(edf['bowler'], edf['economy_rate'])
plt.show()
```



```
In [82]: edf = df.groupby('bowler').agg(
            total=('runs_of_bat', 'sum'),
            balls=('runs_of_bat', 'count')
        ).reset_index()

edf = edf[edf['balls'] >= 18]
edf['economy_rate'] = (edf['total']/edf['balls']) * 6

edf = edf.sort_values('economy_rate').head(10)
edf
```

Out[82]:

	bowler	total	balls	economy_rate
102	Saurabh Dubey	92	85	6.494118
109	Sunil Narine	340	309	6.601942
59	Madhav Tiwari	78	68	6.882353
32	Dilshan Madushanka	32	27	7.111111
8	Anrich Nortje	32	27	7.111111
44	Jason Holder	284	236	7.220339
39	Harpreet Brar	59	49	7.224490
45	Jasprit Bumrah	385	317	7.287066
22	Bhuvneshwar Kumar	487	391	7.473146
7	Akeal Hosein	194	151	7.708609

In [96]:

```
# Exclude wides and no-balls from balls bowled
legal_balls_bowl = df.loc[(df['wide'] == 0) & (df['noballs'] == 0)]

edf_eco = legal_balls_bowl.groupby('bowler').agg(
    total = ('runs_of_bat', 'sum'),
    balls = ('runs_of_bat', 'count')
).reset_index()

edf_eco = edf_eco.loc[edf_eco['balls'] >= 18]
edf_eco['economy_rate'] = ((edf_eco['total'] / edf_eco['balls']) * 6).round(2)
edf_eco = edf_eco.sort_values('economy_rate').head(10)
edf_eco
```

Out[96]:

	bowler	total	balls	economy_rate
109	Sunil Narine	340	309	6.60
102	Saurabh Dubey	92	77	7.17
39	Harpreet Brar	59	48	7.38
44	Jason Holder	284	230	7.41
45	Jasprit Bumrah	375	294	7.65
22	Bhuvneshwar Kumar	483	378	7.67
59	Madhav Tiwari	78	60	7.80
7	Akeal Hosein	194	148	7.86
32	Dilshan Madushanka	32	24	8.00
8	Anrich Nortje	32	24	8.00

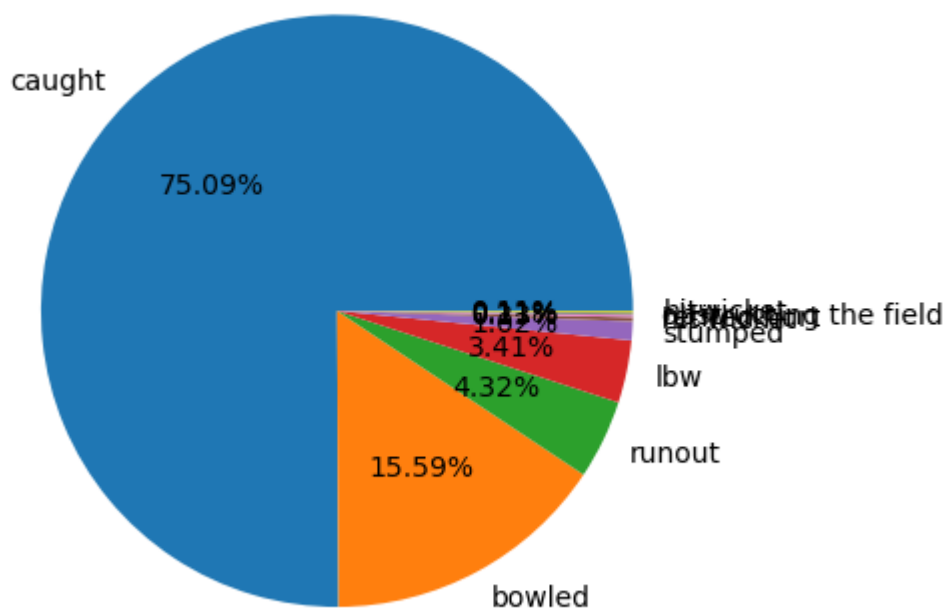
Wicket type


```
In [84]: edf = df['wicket_type'].value_counts().reset_index()
edf
```

```
Out[84]:
```

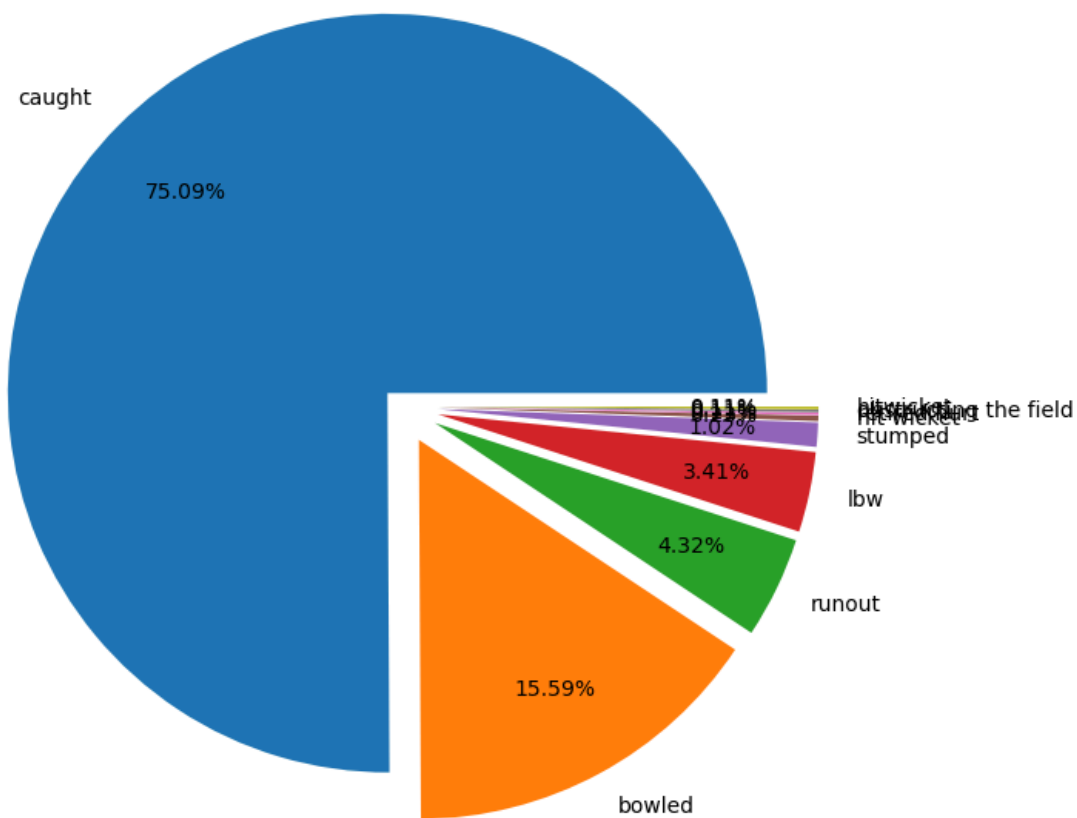
	wicket_type	count
0	caught	660
1	bowled	137
2	runout	38
3	lbw	30
4	stumped	9
5	hit wicket	2
6	retired hurt	1
7	obstructing the field	1
8	hitwicket	1

```
In [85]: plt.pie(edf['count'], labels=edf['wicket_type'], autopct='%0.2f%%')
plt.show()
```

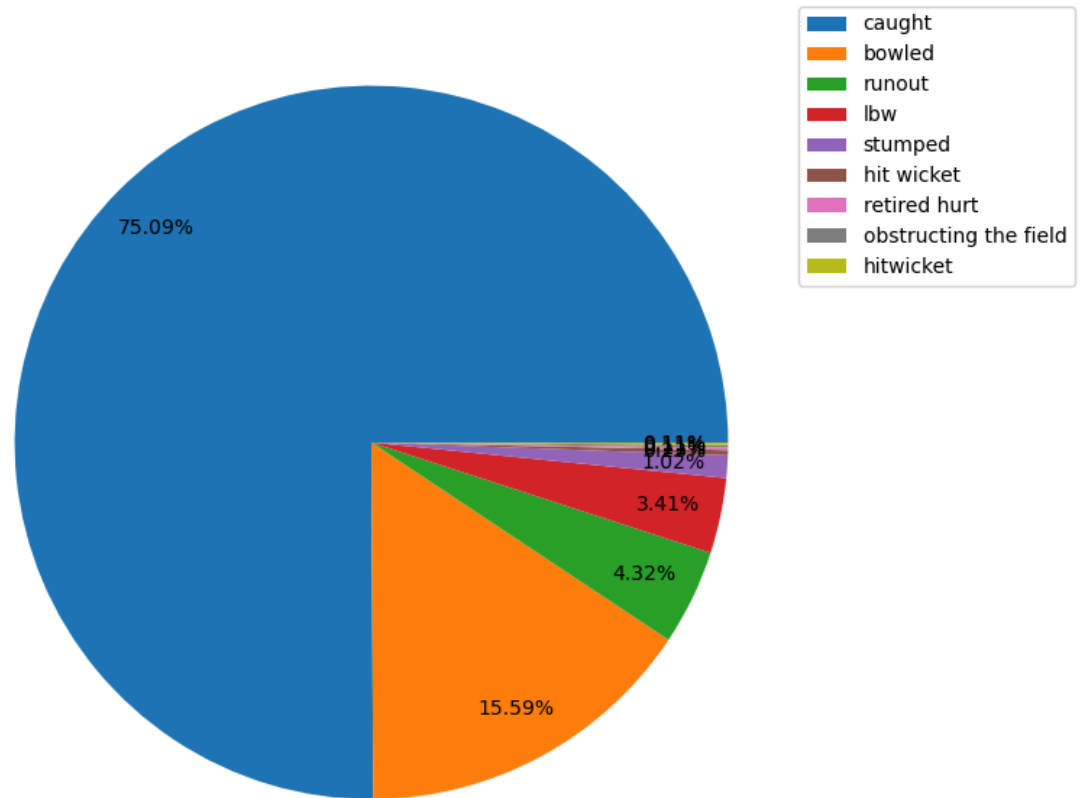


```
In [91]: explode = (0.05, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1) # adjust count to match n

plt.figure(figsize=(10, 8))
plt.pie(edf['count'],
        labels=edf['wicket_type'],
        autopct='%0.2f%',
        explode=explode,
        pctdistance=0.75,
        labeldistance=1.1)
plt.show()
```



```
In [86]: plt.figure(figsize=(10, 8)) # use legend for clean
plt.pie(edf['count'],
        autopct='%0.2f%',
        pctdistance=0.85)
plt.legend(edf['wicket_type'], loc='upper right', bbox_to_anchor=(1.3, 1))
plt.show()
```



avarage run per over

```
In [93]: df['over_number'] = df['over'].astype(int)

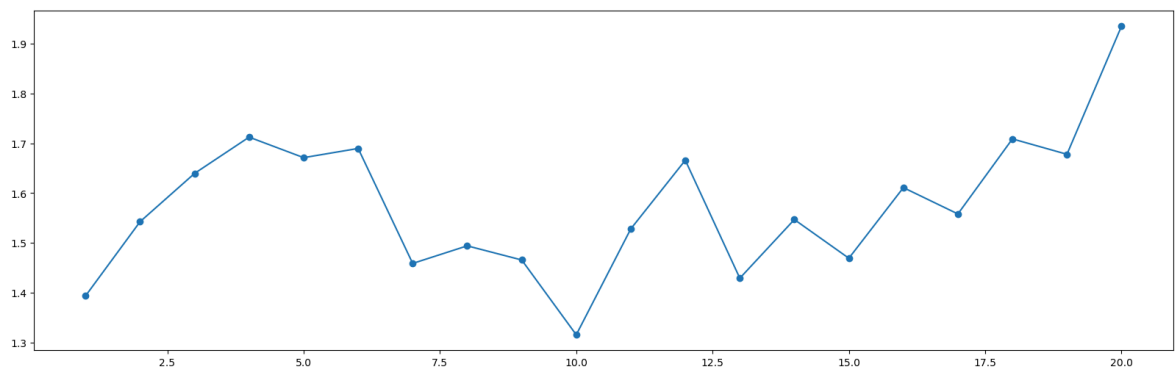
edf = df.groupby('over_number').agg(
    avg_runs=('Total_runs', 'mean')
).reset_index()

edf
```

Out[93]:

	over_number	avg_runs
0	1	1.393425
1	2	1.542427
2	3	1.639434
3	4	1.712254
4	5	1.671053
5	6	1.689503
6	7	1.458564
7	8	1.493896
8	9	1.465479
9	10	1.315205
10	11	1.527930
11	12	1.665919
12	13	1.428889
13	14	1.546682
14	15	1.468997
15	16	1.610855
16	17	1.557945
17	18	1.708690
18	19	1.677807
19	20	1.935000

```
In [94]: plt.figure(figsize=(20,6))  
plt.plot(edf['over_number'], edf['avg_runs'], marker='o')  
plt.show()
```



In []: